

Algorithmique et programmation : 20 exercices corrigés

maths 1re spé — du geste seul au problème de contrôle

Vingt exercices, du geste seul au problème de contrôle : l'affectation simultanée, les listes (en extension, par ajouts, en compréhension), leurs indices et leur parcours, la fonction qui renvoie une valeur, la boucle de seuil sur une suite, la simulation d'un jeu. Des truites dans un lac, la pluie d'une semaine, une randonnée, deux plateformes qui se disputent des abonnés, la racine de 2 par la méthode de Newton. Chaque corrigé montre la trace du programme ou la liste dessinée case par case. Un rappel de cours avant chaque niveau.

★ Un seul geste : exercices 1 à 8 niveau 1

Un programme par exercice : le suivre ligne à ligne, en notant les valeurs.

Rappel de cours

- « $a, b = b, a + b$ » est une affectation SIMULTANÉE : Python calcule d'abord les deux valeurs de droite, avec les anciennes valeurs, puis il les range.
- Une LISTE s'écrit entre crochets : « $L = [5, 8, 2]$ ». Les indices commencent à 0 : $L[0]$ est le premier élément, $L[-1]$ le dernier, $\text{len}(L)$ le nombre d'éléments.
- Trois façons de créer une liste : en EXTENSION « $[3, 6, 9]$ » ; par AJOUTS avec « $L.append(x)$ » dans une boucle ; en COMPRÉHENSION « $[3*k \text{ for } k \text{ in range}(1, 4)]$ ».
- Une fonction RENVOIE son résultat avec « return ». « print » l'affiche seulement : la fonction renvoie alors None, une valeur vide.

- 1 On exécute « $a, b = 1, 1$ », puis trois fois la ligne « $a, b = b, a + b$ ».
 - a) Que valent a et b à la fin ? Faire la trace.
 - b) Un élève remplace chaque ligne « $a, b = b, a + b$ » par les deux lignes « $a = b$ » puis « $b = a + b$ ». Que valent alors a et b à la fin ?

2 Voici trois façons de créer une liste L.

A : « `L = [3, 6, 9, 12]` »

B : « `L = []` » puis « `for k in range(4): L.append(3 * k)` »

C : « `L = [3*k for k in range(1, 5)]` »

a) Donner la liste obtenue par chaque programme.

b) Lesquels donnent la même liste ? Corriger celui qui diffère.

3 On a créé la liste « `L = [5, 8, 2, 9, 4]` », dessinée ci-dessous avec ses indices.

a) Que valent `L[0]`, `L[3]`, `L[-1]` et `len(L)` ?

b) Que contient L après l'instruction « `L[1] = 7` » ?

c) Que se passe-t-il si l'on demande `L[5]` ?

L					
	0	1	2	3	4
	5	8	2	9	4

4 Voici les températures maximales d'une semaine de mai (en °C), rangées dans une liste T, et un programme.

a) Faire la trace de x et de m.

b) Qu'affiche le programme ? Que calcule-t-il ?

c) Pourquoi part-on de « `m = T[0]` » plutôt que de « `m = 0` » ? Penser à une semaine d'hiver très froide.

```
T = [12, 15, 9, 17, 14]
m = T[0]
for x in T:
    if x > m:
        m = x
print(m)
```

5 La suite (u_n) est définie par $u_0 = 5$ et $u_{n+1} = 2u_n - 3$. On exécute « `u = 5` », puis « `for k in range(4): u = 2 * u - 3` ».

a) Faire la trace de k et de u.

b) Quel terme de la suite u contient-il à la fin ?

6 Cette fonction dit combien de racines réelles a le trinôme $ax^2 + bx + c$.

a) Que renvoient `nb_racines(1, -2, 1)`, `nb_racines(2, 3, -5)` et `nb_racines(1, 1, 1)` ?

b) Pourquoi écrit-on « `==` » à la ligne 5, et non « `=` » ?

```
def nb_racines(a, b, c):  
    d = b**2 - 4*a*c  
    if d > 0:  
        return 2  
    elif d == 0:  
        return 1  
    else:  
        return 0
```

7 On exécute « `n = 0` », « `s = 0` », puis « `while s <= 30:` » suivi, en retrait, des deux lignes « `n = n + 1` » et « `s = s + n` ».

On affiche enfin `n` et `s`.

a) Faire la trace de `n` et de `s`.

b) Qu'affiche le programme ? Traduire le résultat par une phrase sur la somme $1 + 2 + \dots + n$.

8 Voici deux fonctions presque identiques.

a) Que vaut « `f(4)` » ?

b) On exécute « `y = f(4) + 1` ». Que vaut `y` ?

c) On exécute « `z = g(4) + 1` ». Le programme affiche 5, puis s'arrête sur une erreur. Expliquer.

```
def f(x):  
    return x**2 - 3*x + 1  
  
def g(x):  
    print(x**2 - 3*x + 1)
```

★★ Type devoir : exercices 9 à 16 niveau 2

Plusieurs étapes, comme au contrôle : lire un programme, le faire tourner à la main, en écrire un morceau.

Rappel de cours

- Parcourir une liste : « `for x in L` » donne les ÉLÉMENTS ; « `for i in range(len(L))` » donne les INDICES, et `L[i]` l'élément d'indice `i`.
- Recherche de SEUIL : « `while u <= S` » répète tant que le seuil `S` n'est pas dépassé. Le compteur `n` donne, à la sortie, le premier rang où il l'est.
- Une fonction peut en appeler une autre : on découpe un problème en petites fonctions, chacune facile à tester.
- Les nombres à virgule (float) sont arrondis en binaire : « `0.1` » n'est pas exact en machine, et les petites erreurs s'accumulent.

9 La suite (u_n) est définie par $u_0 = 1$ et $u_{n+1} = u_n + n$.

Programme A : « $u = 1$ », « $L = [u]$ », puis « for n in range(5): » suivi, en retrait, de « $u = u + n$ » et « $L.append(u)$ ».

Programme B : « $M = [1 + n*(n-1)//2 \text{ for } n \text{ in range}(6)]$ ».

- Faire la trace du programme A. Que contient L à la fin ?
- Que contient M ? Qu'est-ce que cela suggère ?
- Placer les termes dans un repère. La suite est-elle arithmétique ?

10 La liste mm contient la pluie tombée chaque jour d'une semaine, en millimètres.

- Que renvoie « jours_secs(mm) » ? Que compte la fonction ?
- Écrire une fonction « moyenne(L) » qui renvoie la moyenne des éléments de L, en parcourant la liste avec une boucle.
- Que renvoie « moyenne(mm) » ?

```
mm = [0, 12, 3, 0, 25, 9, 0]

def jours_secs(L):
    n = 0
    for x in L:
        if x == 0:
            n = n + 1
    return n
```

11 Sur une randonnée, un GPS relève l'altitude (en m) tous les deux kilomètres : « alt = [320, 450, 610, 580, 720] ».

- Que contient la liste « e = [alt[i+1] - alt[i] for i in range(len(alt) - 1)] » ? Que représente-t-elle ?
- Le DÉNIVELÉ POSITIF est la somme des montées. Le calculer à partir de e.
- Pourquoi « range(len(alt) - 1) », et pas « range(len(alt)) » ?

12 Dans un lac, on modélise le nombre de truites (en milliers) par la suite $u_0 = 1$ et $u_{n+1} = 0,6 u_n + 3$: chaque année, 40 % des truites disparaissent et l'on en réintroduit 3 000.

- Qu'affiche le programme ? Interpréter.
- Que se passerait-il si l'on remplaçait 7 par 8 dans la condition du « while » ?

```
u = 1
n = 0
while u <= 7:
    u = 0.6 * u + 3
    n = n + 1
print(n)
```

13 Pour approcher le nombre dérivé de $f(x) = x^2$ en 1, on calcule des pentes de sécantes de plus en plus proches.

- a) Que calcule « `taux(a, h)` » ? Pourquoi cette fonction appelle-t-elle `f` ?
- b) Python affiche « `[3.0, 2.1000000000000002, 2.0100000000000007]` ». Retrouver ces valeurs par le calcul, et expliquer les derniers chiffres.
- c) Quelle valeur de $f'(1)$ ces pentes suggèrent-elles ?

```
def f(x):  
    return x**2  
  
def taux(a, h):  
    d = f(a + h) - f(a)  
    return d / h  
  
L = [taux(1, h) for h in  
      [1, 0.1, 0.01]]  
print(L)
```

14 Une population de bactéries y vérifie $y' = y$ et $y(0) = 1$: sa vitesse de croissance est égale à sa taille. La méthode d'Euler avance à petits pas h en suivant la tangente : $y(x + h) \approx y(x) + h \times y(x)$.

- a) Faire la trace du programme. Qu'affiche-t-il ?
- b) Comparer avec la valeur exacte $y(1) = e \approx 2,718$. Comment améliorer l'approximation ?
- c) Avec « `h = 0.1` », la boucle tourne 11 fois au lieu de 10. Pourquoi ?

```
x = 0  
y = 1  
h = 0.25  
while x < 1:  
    y = y + h * y  
    x = x + h  
print(y)
```

15 On veut la liste des nombres premiers inférieurs à 20. Un nombre premier a exactement deux diviseurs : 1 et lui-même.

- a) Que renvoie « `diviseurs(12)` » ?
- b) Que renvoient « `est_premier(13)` » et « `est_premier(1)` » ?
- c) Que contient la liste P ?

```
def diviseurs(n):  
    L = []  
    for d in range(1, n + 1):  
        if n % d == 0:  
            L.append(d)  
    return L  
  
def est_premier(n):  
    k = len(diviseurs(n))  
    return k == 2  
  
P = [n for n in range(20)  
     if est_premier(n)]
```

16 Un escargot parcourt 1 m la première heure, puis, chaque heure, la moitié de la distance de l'heure précédente. On exécute « `S = 0` », « `u = 1` », puis « `for k in range(5):` » suivi, en retrait, de « `S = S + u` » et « `u = u / 2` ».

- a) Faire la trace de S et de u. Que vaut S à la fin ? Que représente-t-il ?
- b) Un élève inverse les deux lignes de la boucle. Que vaut alors S ?
- c) L'escargot atteindra-t-il 2 m ?

☆☆☆ Problèmes : exercices 17 à 20 niveau 3

Des situations complètes : on lit le programme, on le fait tourner à la main, on interprète ce qu'il affiche.

Rappel de cours

- Un programme se lit en trois temps : ce que contiennent les variables au départ, ce que fait UN tour de boucle, quand on s'arrête.
- « `from random import randint` » puis « `randint(1, 6)` » : un entier au hasard entre 1 et 6, bornes COMPRISES. La moyenne des gains sur un grand nombre de parties approche l'espérance.
- Une suite se programme par sa relation de récurrence : une variable u, remplacée à chaque tour par le terme suivant. Pour garder tous les termes, on les ajoute à une liste.

17 Le jeu de dé de la fête

À une fête de village, on lance un dé. Avec un 6, on gagne 5 € ; avec un 4 ou un 5, on gagne 1 € ; sinon, on perd 2 €.

- Le gain d'une partie est une variable aléatoire X . Donner sa loi et calculer son espérance.
- Que renvoie « partie() » ? Que calcule « moyenne(N) » ?
- Un essai a donné : « moyenne(10) » renvoie « -0.8 », « moyenne(100) » renvoie « 0.0 », « moyenne(10000) » renvoie « 0.1703 ». Commenter.
- Le jeu est-il favorable au joueur ?

```
from random import randint

def partie():
    d = randint(1, 6)
    if d == 6:
        return 5
    elif d >= 4:
        return 1
    else:
        return -2

def moyenne(N):
    L = [partie()
         for k in range(N)]
    return sum(L) / N
```

18 Qui aura le plus d'abonnés ?

Deux plateformes lancent leur service le même mois. On modélise leurs abonnés, en milliers : A part de 5 et gagne 0,5 par mois ; B part de 1 et augmente de 25 % par mois. On note a_n et b_n ces nombres au bout de n mois.

- Exprimer a_n et b_n en fonction de n .
- Qu'affiche le programme ? Interpréter.
- Pourquoi la condition est-elle « $b \leq a$ », et pas « $b < a$ » ?
- Modifier le programme pour qu'il range aussi les valeurs successives de b dans une liste LB.

```
a = 5
b = 1
n = 0
while b <= a:
    a = a + 0.5
    b = b * 1.25
    n = n + 1
print(n)
```

19 Les kilomètres d'une course

Une coureuse chronomètre chaque kilomètre d'une course de 5 km. Les temps, en secondes, sont rangés dans la liste T.

- Que renvoie « plus_rapide(T) » ? Quel kilomètre a-t-elle couru le plus vite ?
- Que contient la liste C à la fin ? Que représente C[-1] ?
- Calculer son allure moyenne, en minutes par kilomètre.

```
T = [312, 305, 298, 301, 284]

def plus_rapide(T):
    i_min = 0
    for i in range(len(T)):
        if T[i] < T[i_min]:
            i_min = i
    return i_min

C = [T[0]]
for i in range(1, len(T)):
    C.append(C[-1] + T[i])
```

20 La racine de 2 par la méthode de Newton

On cherche la solution positive de $x^2 - 2 = 0$, c'est-à-dire $\sqrt{2}$. On part de $x_0 = 1$ et, à chaque étape, on remplace x par l'abscisse du point où la tangente à la courbe de $f(x) = x^2 - 2$ coupe l'axe des abscisses : $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$.

- Écrire l'équation de la tangente en $x_0 = 1$ et retrouver $x_1 = 1,5$.
- Expliquer le rôle de fp et de la liste L. Pour « newton(1, 4) », Python affiche « [1, 1.5, 1.4166666666666667, 1.4142156862745099, 1.4142135623746899] » : vérifier x_2 par le calcul.
- Sachant que $\sqrt{2} = 1,41421356237 \dots$, combien de décimales justes a chaque terme ? Que remarque-t-on ?

```
def f(x):
    return x**2 - 2

def fp(x):
    return 2*x

def newton(x, n):
    L = [x]
    for k in range(n):
        x = x - f(x) / fp(x)
        L.append(x)
    return L
```

Corrigés : algorithmique et programmation, 20 exercices

Ces pages se détachent de la feuille d'exercices. Chaque étape est écrite, avec son pourquoi.

1. a) Dans « $a, b = b, a + b$ », Python calcule D'ABORD les deux valeurs de droite avec les anciennes valeurs, puis il les range.

Départ : a vaut 1 et b vaut 1.

1er passage : a reçoit 1 (l'ancien b), b reçoit $1 + 1 = 2$.

2e passage : a reçoit 2, b reçoit $1 + 2 = 3$.

3e passage : a reçoit 3, b reçoit $2 + 3 = 5$.

À la fin, a vaut 3 et b vaut 5. On reconnaît les nombres de Fibonacci : 1, 1, 2, 3, 5...

b) Avec deux lignes, « $b = a + b$ » utilise le NOUVEAU a .

1er passage : a vaut 1, puis b vaut $1 + 1 = 2$. 2e : a vaut 2, puis b vaut $2 + 2 = 4$. 3e : a vaut 4, puis b vaut $4 + 4 = 8$.

À la fin, a vaut 4 et b vaut 8 : ce n'est plus la même suite.

⊖ Le piège : croire que les deux écritures font la même chose. L'affectation simultanée évite d'écraser une valeur dont on a encore besoin.

étape	a	b
0	1	1
1	1	2
2	2	3
3	3	5

2. a) A est écrite en EXTENSION : L vaut [3, 6, 9, 12].

B part d'une liste vide et AJOUTE $3k$ à la fin, pour $k = 0, 1, 2, 3$: L vaut [0, 3, 6, 9].

C est écrite en COMPRÉHENSION, pour $k = 1, 2, 3, 4$: L vaut [3, 6, 9, 12].

b) A et C donnent la même liste ; B commence à 0.

Pour corriger B , on fait partir k de 1 : « `for k in range(1, 5): L.append(3 * k)` ».

⊖ Le piège : « `range(4)` » commence à 0 et s'arrête à 3. Il donne bien quatre valeurs, mais pas celles qu'on croit.

3. a) Les indices commencent à 0 : $L[0]$ est le PREMIER élément, 5.

$L[3]$ est le quatrième élément : 9.

$L[-1]$ est le DERNIER élément : 4.

$\text{len}(L)$ est le nombre d'éléments : 5.

b) « $L[1] = 7$ » remplace l'élément d'indice 1, c'est-à-dire le deuxième : L devient [5, 7, 2, 9, 4].

c) Les indices vont de 0 à 4 : $L[5]$ n'existe pas. Python s'arrête sur une erreur (IndexError).

⊖ Le piège : croire que $L[3]$ est le troisième élément. L'indice est toujours décalé de 1 par rapport au rang.

4. a) « `for x in T` » donne à x chaque ÉLÉMENT de la liste, dans l'ordre : 12, 15, 9, 17, 14.

m vaut d'abord 12. Avec $15 > 12$, m devient 15. Avec 9, rien ne change. Avec $17 > 15$, m devient 17. Avec 14, rien ne change.

b) Le programme affiche 17 : c'est le MAXIMUM de la liste.

c) Si toutes les températures sont négatives, par exemple [-8, -3, -11], « $m = 0$ » donnerait 0, qui n'est même pas dans la liste. En partant de $T[0]$, m est toujours une vraie valeur de la liste.

★ Pour trouver le minimum, on garde le même programme et on remplace « $>$ » par « $<$ ».

5. a) « `range(4)` » donne $k = 0, 1, 2, 3$: la boucle fait 4 tours.

À chaque tour, u est remplacé par $2u - 3$: $2 \times 5 - 3 = 7$, puis $2 \times 7 - 3 = 11$, puis 19, puis 35.

b) Au départ, u contient u_0 ; après un tour, u_1 ; après 4 tours, u_4 .

À la fin, u vaut $35 = u_4$.

⊖ Le piège : répondre u_3 , parce que le dernier k vaut 3. Ce qui compte, c'est le NOMBRE de tours : 4 tours font avancer de 4 rangs.

6. a) La fonction calcule le discriminant d , puis teste les cas DANS L'ORDRE et renvoie le premier qui convient.
 $\text{nb_racines}(1, -2, 1) : d = (-2)^2 - 4 \times 1 \times 1 = 0$. « $d > 0$ » est faux, « $d == 0$ » est vrai : elle renvoie 1.
 $\text{nb_racines}(2, 3, -5) : d = 9 + 40 = 49 > 0$: elle renvoie 2.
 $\text{nb_racines}(1, 1, 1) : d = 1 - 4 = -3$. Les deux tests sont faux : on arrive au « else », elle renvoie 0.
b) « $==$ » COMPARE deux valeurs et donne True ou False. « $=$ » RANGE une valeur : dans un « if », Python refuserait la ligne.
 ★ Dès qu'un « return » est exécuté, la fonction s'arrête : les lignes suivantes ne sont pas lues.

7. a) On teste la condition AVANT chaque tour : on continue tant que s est inférieur ou égal à 30.
 n vaut 1, 2, 3... et s ajoute n à chaque tour : 1, 3, 6, 10, 15, 21, 28.
 Avec $s = 28$, la condition « $28 \leq 30$ » est vraie : encore un tour. n devient 8 et s devient 36.
 Maintenant « $36 \leq 30$ » est faux : on sort.
b) Le programme affiche « 8 36 ».
 Phrase : 8 est le plus petit entier n tel que $1 + 2 + \dots + n > 30$; cette somme vaut alors 36.
 ⚠ Le piège : s'arrêter à 28, « la dernière somme sous 30 ». La boucle s'arrête quand la condition devient FAUSSE, donc juste APRÈS avoir dépassé 30.

8. a) $f(4) = 4^2 - 3 \times 4 + 1 = 16 - 12 + 1 = 5$.
b) f RENVOIE son résultat : « $f(4)$ » vaut 5, qu'on peut réutiliser dans un calcul. y vaut $5 + 1 = 6$.
c) g AFFICHE le nombre 5 à l'écran, mais ne renvoie rien : en Python, « $g(4)$ » vaut None, une valeur vide.
 « $\text{None} + 1$ » n'a pas de sens : Python s'arrête (TypeError).
 ★ « print » montre un résultat à l'humain ; « return » le donne au programme. Une fonction mathématique se programme avec « return ».
 ⚠ Le piège : croire que g fait comme f parce qu'on voit « 5 » à l'écran.

9. a) L commence avec $u_0 = 1$. À chaque tour, on calcule le terme suivant, puis on l'AJOUTE à la fin de L .
 $n = 0$: u devient $1 + 0 = 1$. $n = 1$: $1 + 1 = 2$. $n = 2$: $2 + 2 = 4$. $n = 3$: $4 + 3 = 7$. $n = 4$: $7 + 4 = 11$.
 L vaut $[1, 1, 2, 4, 7, 11]$: ce sont $u_0, \dots, u_5$, soit six termes pour cinq tours.
b) M est écrite en compréhension : pour n de 0 à 5, on calcule $1 + \frac{n(n-1)}{2}$ (« // » est la division entière, sans perte ici car $n(n-1)$ est toujours pair). M vaut aussi $[1, 1, 2, 4, 7, 11]$.
 Les deux listes coïncident : on peut CONJECTURER que $u_n = 1 + \frac{n(n-1)}{2}$ pour tout n . Six termes ne le démontrent pas, mais le rendent plausible.
c) Les points ne sont pas alignés : la suite n'est pas arithmétique (les écarts 0, 1, 2, 3, 4 ne sont pas constants). Ils sont sur la parabole $y = 0,5x^2 - 0,5x + 1$.
 ⚠ Le piège : oublier que L contient déjà u_0 avant la boucle. Cinq tours donnent six termes.

10. a) La boucle parcourt les ÉLÉMENTS de la liste. À chaque jour sans pluie ($x == 0$), le compteur n augmente de 1.
 Les jours à 0 mm sont le 1er, le 4e et le 7e : la fonction renvoie 3. Elle compte les jours SECS.
b) On additionne les éléments dans une variable s , puis on divise par le nombre d'éléments, $\text{len}(L)$: voir le programme.
c) $s = 0 + 12 + 3 + 0 + 25 + 9 + 0 = 49$ et $\text{len}(\text{mm})$ vaut 7 : « $\text{moyenne}(\text{mm})$ » renvoie « 7.0 », soit 7 mm par jour en moyenne.
 ★ « 7.0 » et non « 7 » : la division « / » donne toujours un float.
 ⚠ Le piège au b) : diviser par 7 écrit à la main. Avec « $\text{len}(L)$ », la fonction marche pour une liste de n'importe quelle longueur.

11. a) $\text{len}(\text{alt})$ vaut 5, donc « $\text{range}(4)$ » : i prend les valeurs 0, 1, 2, 3.
 Pour chaque i , on calcule l'écart entre deux relevés voisins : $450 - 320 = 130$; $610 - 450 = 160$; $580 - 610 = -30$; $720 - 580 = 140$.
 e vaut $[130, 160, -30, 140]$: ce sont les VARIATIONS d'altitude sur chaque tronçon de 2 km. Le -30 est une descente.
b) On ajoute les écarts positifs : $130 + 160 + 140 = 430$ m.
 ⚠ Ce n'est pas $720 - 320 = 400$ m : la descente de 30 m a dû être remontée.
c) Avec « $\text{range}(5)$ », le dernier tour prendrait $i = 4$ et demanderait $\text{alt}[5]$, qui n'existe pas : erreur IndexError. Cinq relevés donnent

seulement quatre écarts.

➡ Le piège : oublier qu'une liste de n éléments a ses indices de 0 à $n - 1$.

12. a) On calcule les termes tant qu'ils ne dépassent pas 7 : $u_1 = 0,6 \times 1 + 3 = 3,6$; $u_2 = 5,16$; $u_3 = 6,096$; $u_4 \approx 6,66$; $u_5 \approx 6,99$; $u_6 \approx 7,20$.

$u_5 \approx 6,99$ est encore inférieur ou égal à 7 : un tour de plus. $u_6 > 7$: on sort. Le programme affiche 6.

Interprétation : c'est au bout de 6 ans que le lac compte, pour la première fois, plus de 7 000 truites.

b) Les termes montent vers $7,5$ sans jamais le dépasser : si $u_n < 7,5$, alors $u_{n+1} = 0,6 u_n + 3 < 0,6 \times 7,5 + 3 = 7,5$.

Donc u reste toujours sous 8 : la condition « $u \leq 8$ » reste vraie, et la boucle ne s'arrête JAMAIS.

➡ Le piège : lancer une recherche de seuil sans vérifier que le seuil peut être atteint. Sur le dessin, les points franchissent la droite $y = 7$, mais se tassent bien sous la droite $y = 8$.

13. a) « $\text{taux}(a, h)$ » calcule $\frac{f(a+h) - f(a)}{h}$: la pente de la sécante qui joint les points de la courbe d'abscisses a et $a + h$.

Elle appelle f au lieu de recopier la formule : pour changer de fonction, on ne modifie QUE f . C'est l'intérêt de découper en fonctions.

b) $\frac{f(1+h) - f(1)}{h} = \frac{(1+h)^2 - 1}{h} = \frac{2h + h^2}{h} = 2 + h$.

Pour $h = 1$: 3 ; pour $h = 0,1$: 2,1 ; pour $h = 0,01$: 2,01.

Les derniers chiffres de « 2.100000000000002 » viennent des arrondis du calcul en binaire : $0,1$ ne s'écrit pas exactement en machine.

c) Les pentes $2 + h$ se rapprochent de 2 quand h rétrécit : on conjecture $f'(1) = 2$, ce que confirme la formule $f'(x) = 2x$.

★ Sur le dessin : la sécante pour $h = 1$ (en vert, pente 3) pivote autour du point $(1; 1)$ vers la tangente (en orange, pente 2).

14. a) À chaque tour, y est multiplié par $1 + h = 1,25$ et x avance de $0,25$.

x : 0,25 ; 0,5 ; 0,75 ; 1. y : 1,25 ; 1,5625 ; 1,953125 ; 2,44140625.

Avec $x = 1$, « $1 < 1$ » est faux : on sort après 4 tours. Le programme affiche $2,44140625 = 1,25^4$.

b) Euler donne une valeur trop petite : la courbe s'incurve vers le haut, et les segments de tangente restent en dessous.

Pour faire mieux, on diminue le pas h : plus de tours, mais des segments plus courts, qui collent mieux à la courbe.

c) En machine, $0,1$ n'est pas exact. Après dix ajouts, x vaut $0,9999999999999999$, un peu moins que 1 : « $x < 1$ » est encore vrai, et la boucle fait un 11e tour.

Le programme affiche alors $1,1^{11} \approx 2,85$ au lieu de $1,1^{10} \approx 2,59$.

➡ Le piège : arrêter une boucle sur une comparaison de float. Plus sûr : compter les tours, avec « `for k in range(10)` ».

15. a) `diviseurs(n)` parcourt d de 1 à n (« `range(1, n + 1)` », car la borne de droite est exclue) et AJOUTE d à L quand le reste « $n \% d$ » est nul.

`diviseurs(12)` renvoie `[1, 2, 3, 4, 6, 12]`.

b) `est_premier` APPELLE `diviseurs` et compte les diviseurs obtenus. 13 en a deux, `[1, 13]` : True.

1 n'en a qu'un, `[1]` : False. C'est voulu : 1 n'est pas un nombre premier.

c) `P` garde les n de 0 à 19 pour lesquels `est_premier(n)` est vrai : `P` vaut `[2, 3, 5, 7, 11, 13, 17, 19]`.

0 est écarté aussi : `diviseurs(0)` est la liste vide, qui a 0 élément.

★ Deux petites fonctions, chacune facile à tester, font un programme sûr : c'est le découpage en fonctions.

➡ Le piège au a) : « `range(1, n)` », qui oublie n lui-même. Alors 13 n'aurait qu'un diviseur et ne serait plus premier.

16. a) u est la distance parcourue pendant l'heure en cours ; S additionne ces distances.

Tour 1 : S reçoit $0 + 1 = 1$, puis u devient $0,5$. Tour 2 : S vaut $1,5$, u vaut $0,25$. Puis S vaut $1,75$; $1,875$; $1,9375$.

À la fin, S vaut $1,9375$: la distance totale parcourue en 5 heures, en mètres.

b) Si l'on divise u par 2 AVANT de l'ajouter, la première heure compte pour $0,5$: S vaut $0,5 + 0,25 + 0,125 + 0,0625 + 0,03125 = 0,96875$.

C'est faux : l'ordre des lignes dans une boucle compte.

c) Après chaque heure, il manque exactement la distance de l'heure qui vient de s'écouler : $2 - 1,9375 = 0,0625$. L'escargot s'approche de 2 m, sans jamais les atteindre.

★ Ici, tous les calculs sont exacts en machine : $0,5$, $0,25$... s'écrivent parfaitement en binaire.

17. Le jeu de dé de la fête

$$a) P(X = 5) = \frac{1}{6}; P(X = 1) = \frac{2}{6}; P(X = -2) = \frac{3}{6}.$$

$$E(X) = 5 \times \frac{1}{6} + 1 \times \frac{2}{6} - 2 \times \frac{3}{6} = \frac{5 + 2 - 6}{6} = \frac{1}{6} \approx 0,17 \text{ €}.$$

b) « partie() » tire un dé et RENVOIE le gain d'une partie : 5, 1 ou -2, avec les probabilités de la loi de X .

« moyenne(N) » APPELLE partie() N fois, range les N gains dans la liste L (en compréhension), puis renvoie leur moyenne : le gain moyen sur N parties.

c) Sur 10 parties, on a perdu en moyenne 0,80 € par partie ; sur 100, on n'a rien gagné. Sur 10 000 parties, le gain moyen vaut 0,1703 €, très proche de $E(X) \approx 0,1667$.

Plus N est grand, plus la moyenne se rapproche de l'espérance : c'est la loi des grands nombres.

d) $E(X) > 0$: en moyenne, le joueur gagne environ 17 centimes par partie. Le jeu lui est légèrement favorable, et fera perdre de l'argent à l'organisateur sur un grand nombre de parties.

🚫 Le piège : juger le jeu sur 10 parties. Un petit nombre de parties fluctue beaucoup : ici, il faisait croire à un jeu perdant.

18. Qui aura le plus d'abonnés ?

a) A gagne la même quantité chaque mois : suite arithmétique, $a_n = 5 + 0,5n$.

B est multipliée par 1,25 chaque mois : suite géométrique, $b_n = 1,25^n$.

b) La boucle tourne tant que B n'a pas dépassé A. Au bout de 10 mois : $a_{10} = 10$ et $b_{10} \approx 9,31$. Au bout de 11 mois : $a_{11} = 10,5$ et $b_{11} \approx 11,64$: on sort.

Le programme affiche 11 : B dépasse A au bout de 11 mois, avec environ 11 640 abonnés contre 10 500.

c) On veut le premier mois où B a STRICTEMENT plus d'abonnés que A. Avec « $b < a$ », une égalité arrêterait la boucle trop tôt (ici, elle ne se produit pas).

d) Avant la boucle, « $LB = [b]$ » ; dans la boucle, après « $b = b * 1.25$ », on ajoute « $LB.append(b)$ ». À la fin, LB contient $b_0, \dots, b_{11}$: douze valeurs.

★ Une croissance en pourcentage finit toujours par dépasser une croissance constante : sur le dessin, la ligne orange de B finit par croiser la droite de A.

19. Les kilomètres d'une course

a) La fonction parcourt les INDICES i de 0 à 4 et retient dans i_min l'indice du plus petit temps rencontré.

$T[4] = 284$ est le plus petit : « $plus_rapide(T)$ » renvoie 4.

L'indice 4 correspond au 5e kilomètre, le dernier, couru en 284 s, soit 4 min 44 s.

🚫 Le piège : répondre « le 4e kilomètre ». L'indice commence à 0 : l'indice 4 est le 5e élément.

b) C commence par $T[0] = 312$; chaque tour ajoute à la fin le dernier total, $C[-1]$, plus le temps suivant.

C vaut $[312, 617, 915, 1216, 1500]$: les temps CUMULÉS au passage de chaque kilomètre.

$C[-1] = 1500$ s est le temps total de la course : 25 minutes.

c) Allure moyenne : $\frac{1500}{5} = 300$ s par kilomètre, soit 5 min par kilomètre.

★ Elle a fini plus vite qu'elle n'a commencé : son dernier kilomètre est le plus rapide.

20. La racine de 2 par la méthode de Newton

a) $f(1) = -1$ et $f'(x) = 2x$, donc $f'(1) = 2$. Tangente : $y = 2(x - 1) - 1 = 2x - 3$.

Elle coupe l'axe des abscisses quand $2x - 3 = 0$: $x_1 = 1,5$. C'est bien $1 - \frac{f(1)}{f'(1)} = 1 - \frac{-1}{2} = 1,5$.

b) fp est la dérivée f' . newton APPELLE f et fp à chaque étape, et range chaque nouvelle valeur dans L : la liste garde tous les termes, de x_0 à x_n .

$$x_2 = 1,5 - \frac{1,5^2 - 2}{2 \times 1,5} = 1,5 - \frac{0,25}{3} = \frac{17}{12} \approx 1,4167.$$

c) $x_1 = 1,5$: aucune décimale juste. $x_2 \approx 1,4167$: 2 décimales. $x_3 \approx 1,414216$: 5 décimales. $x_4 \approx 1,414213562375$: 11 décimales.

Le nombre de décimales justes double à peu près à chaque étape : la méthode est très rapide.

★ Pour $\sqrt{2}$, ce calcul est celui qu'on attribue à Héron d'Alexandrie (Ier siècle) ; la méthode générale porte le nom de Newton (XVIIe siècle).

